```python
# Libraries for parsing and modifying data
# Libraires are collections of pre-written code, built by others to solve common problems.
# Imagine them being a specialized tool, like a microscope or a calculator
import pandas as pd # a library for working with structured data, like a spreadsheet, so we can work with data easily
import os # a library for you to talk to your computer's file system
from lxml import etree # getting information from xml files
from bs4 import BeautifulSoup # read and extract data from .xml files
import matplotlib
import matplotlib.pyplot as plt
import random # generates random numbers, we use it for sampling
import numpy as np # fasr numerical computing

# Libraries from scikit-learn to create topic models
import sklearn
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.decomposition import LatentDirichletAllocation
from sklearn.decomposition import NMF
```

```python
# Set corpus to the folder of files you want to use
corpus = '/home/ec2-user/SageMaker/data/dh340news/'

# Go through the items in corpus one by one, and only keeps XML files
input_files = [
    f for f in os.listdir(corpus)
    if f.lower().endswith(".xml")
]
```

```python
# Select the number of articles to sample
sample_size = 469

# Upper limit of documents is 10000
if sample_size > 10000:
    sample_size = 10000

# Generate a sample of articles
try:
    sample_input_files = input_files[0:sample_size]

except ValueError:
    sample_input_files = input_files

print("Currently sampling", len(sample_input_files), "documents.")
```

```
Currently sampling 469 documents.
```

```python
# Function to strip html tags from text portion
def strip_html_tags(text): # define a function that takes one argument, text
    stripped = BeautifulSoup(text).get_text().replace('\n', ' ').replace('\\', '').strip() # parses the input string as HTML into
    return stripped
# .get_text ():extract only visible text from HTML, remove all tages like <p> <div> etc
# .replace('\n') converts newsline characters into spaces so lines don't run together
# .replace('\\') remove backslash characters
# .strip() trim whitespaces
# Then give us the cleaned string
```

```python
# Retrieve metadata from XML document
# define a function, corpus is the folder path that has all the xml files
# file, the filename of one xml file within that folder
def getxmlcontent(corpus, file, strip_html=True):
    # create 5 variables and they are all none, so if anything is missing in the xml, it doesn't break the function
    goid = title = date = publisher = text = None
    # use try except to catch any potential problems
    try:
        # let's read and parse xml, grab the topmost xml element
        tree = etree.parse(corpus + file)
        root = tree.getroot()

        # search under root(.//) for the first <GOID> element, if find it, store its text content in goid; didn't find it, None
        if root.find('.//GOID') is not None:
            goid = root.find('.//GOID').text
        else:
            goid = None

        # search, <Title>, find it store under title
        if root.find('.//Title') is not None:
            title = root.find('.//Title').text
        else:
            title = None
```

```python
        # same
        if root.find('.//NumericDate') is not None:
            date = root.find('.//NumericDate').text
        else:
            date = None

        # same
        if root.find('.//PublisherName') is not None:
            publisher = root.find('.//PublisherName').text
        else:
            publisher = None

        # same, search for the first <FullText> element, if found it, save text; if no full text, search <HiddenText> or <Text>
        if root.find('.//FullText') is not None:
            text = root.find('.//FullText').text

        elif root.find('.//HiddenText') is not None:
            text = root.find('.//HiddenText').text

        elif root.find('.//Text') is not None:
            text = root.find('.//Text').text

        else:
            text = None
```

```python
        # Strip html from text portion
        if text is not None and strip_html == True:
            text = strip_html_tags(text)

    except Exception as e:
        print(f"Error while parsing file: {e}")

    # return 5-tuple with all extracted fields
    return goid, title, date, publisher, text
```

```python
# Create a dictionary to store key/value pairs, we can convert this to a dataframe with ease later
metadata = {}

# Parse files and fill article_metadata_text list with relevant text
# iterates over each filename in sample_input_files
# then call you function getxmlcontent to handle one xml file, return 5 values
for file in sample_input_files:

    goid, title, date, publisher, text = getxmlcontent(corpus, file, strip_html=True)

    # store what we need, for each goid, we only keep data and text; we still have title and publisher, but they didn't get stored
    metadata[goid] = {}
    metadata[goid]['date'] = date
    metadata[goid]['text'] = text
```

```python
# convert metadata into a pandas DataFrame
# orient='index' means treat directionary key (goid) as the row index, then the nested dicts ({'date':,
# .dropna() drop things that are none
df_metadata = (pd.DataFrame.from_dict(metadata, orient='index')).dropna()

# takes the text column of the DataFrame, make it into plain python list, we need raw_texts
raw_texts = df_metadata['text'].tolist()
```

```python
# Print out length of created list for checking
print("Processed", len(raw_texts), "documents.")
```

```
Processed 467 documents.
```

```python
print("Before deduplication:", len(raw_texts))
dedup_texts = list(dict.fromkeys(raw_texts))
print("After deduplication:",len(dedup_texts))
```

```
Before deduplication: 467
After deduplication: 364
```

```python
#raw_texts = dedup_texts
df_metadata_dedup = df_metadata.drop_duplicates(subset="text").copy()
len(df_metadata), len(df_metadata_dedup)
```

```
(467, 364)
```

```python
raw_texts = df_metadata_dedup["text"].tolist()
```

```
# We are building a bag-of-words vocabulary with scikitlearns' CountVectorizer, we imported this at the beginning
lowercase = False
token_pattern = r'(?u)\b[A-Za-z][A-Za-z0-9]{1,}\b' # use regular expression to say what we think is a token
# 2+ characters; first one must be a letter or
ngram_range = (1,2) # include single words and 2-word phrases ("data center" we treat it as one rather than two)
max_features = 5000 # set upper limit number of unique words, you can change this
max_df = .5 # Maximum doc frequency; drops terms that appear in > 50% of documents.
min_df = .01 # Minimum doc frequency: keeps term that appear in >= 1% of documents, so for us, at least in 4.67 documents

tokenized_data = CountVectorizer(max_df=max_df,
                                 min_df=min_df,
                                 #lowercase=True,
                                 stop_words='english',
                                 ngram_range=ngram_range,
                                 max_features=max_features,
                                 token_pattern=token_pattern)


# Produce document_term matrix, it gives a matrix of shape
# each row = a document
# each column = a feature(word)
doc_mat = tokenized_data.fit_transform(raw_texts)

# For use as column name in plot
# returns a NumPy array of the final feature strings in column order
features = tokenized_data.get_feature_names_out()

print("Number of features: ", len(features))
print("Shape of matrix: ", doc_mat.shape)
```

```
#features[100:200]
features[-60:]
```

```
array(['world', 'world largest', 'worldwide', 'worried', 'worries',
       'worry', 'worse', 'worst', 'worth', 'worth billion', 'wouldn',
       'wright', 'wright said', 'write', 'writer', 'writes', 'writing',
       'written', 'written fridays', 'wrong', 'wrote', 'wrote email',
       'xai', 'xcel', 'yards', 'year according', 'year ago', 'year local',
       'year million', 'year old', 'year period', 'year said',
       'year year', 'years ago', 'years company', 'years data',
       'years old', 'years particularly', 'years said', 'yes', 'yield',
       'york', 'york based', 'york city', 'york times', 'young',
       'young people', 'younger', 'youngkin', 'youtube',
       'youtube tinyurl', 'ypsilanti', 'ypsilanti township', 'zero',
       'zone', 'zoned', 'zoning', 'zoning ordinance', 'zoom', 'zr1'],
      dtype=object)
```

```
# Set Parameters for LDA model
number_of_topics = 8
random_num = 0
```

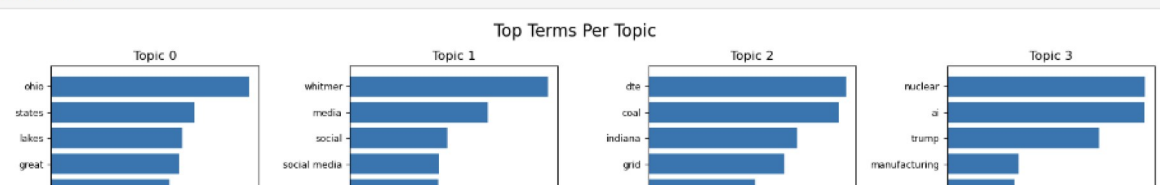
```
# Create LDA model for the term frequency document-term matrix
# Should output general descript about model in the format 'LatentDirichletAllocation(n_components=...)'
lda_model = LatentDirichletAllocation(n_components=number_of_topics,
                                      random_state=random_num)
lda_model.fit(doc_mat)
```

```
LatentDirichletAllocation
LatentDirichletAllocation(n_components=8, random_state=0)
```

```python
# topic to top terms
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os

TOP_N_TERMS = 15
OUT_DIR = "outputresults/"
os.makedirs(OUT_DIR, exist_ok = True)

terms = np.array(tokenized_data.get_feature_names_out())

topic_word = lda_model.components_.copy()
topic_word_norm = topic_word / topic_word.sum(axis=1, keepdims=True)

rows = []
for t in range(topic_word.shape[0]):
    idx = np.argsort(topic_word[t])[::-1][:TOP_N_TERMS]
    for rank, i in enumerate(idx, 1):
        rows.append({
            "topic_id": t,
            "rank": rank,
            "terms": terms[i],
            "weight": float(topic_word_norm[t,i])
        })
topic_top_terms = pd.DataFrame(rows)
topic_top_terms.to_csv(os.path.join(OUT_DIR, "topic_top_terms.csv"), index=False)
topic_top_terms.head(30)
```

```python
df = pd.read_csv("outputresults/topic_top_terms.csv")

top_n = 8
topics = sorted(df["topic_id"].unique())

n_cols = 4
n_rows = (len(topics) + n_cols -1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize = (16, 8))
axes = axes.flatten()

for ax, topic in zip(axes, topics):
    subset = (
        df[df["topic_id"] == topic]
        .sort_values("rank")
        .head(top_n)
        .sort_values("weight")
    )

    ax.barh(subset["terms"], subset["weight"])
    ax.set_title(f"Topic {topic}")
    ax.set_xlabel("Weight")
    ax.tick_params(axis = "y", labelsize = 9)

for ax in axes [len(topics):]:
    ax.axis("off")

fig.suptitle("Top Terms Per Topic", fontsize = 16)
plt.tight_layout()
```

```python
fig.suptitle("Top Terms Per Topic", fontsize = 16)
plt.tight_layout()

plt.savefig(
    "lda_topics_overview.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```
]: #print(fig)
```

```
Figure(1600x800)
```

```
]: doc_topic = lda_model.transform(doc_mat)

   topic_cols = [f"topic_{i}" for i in range(doc_topic.shape[1])]

   df_doc_topic = pd.DataFrame(
       doc_topic,
       columns=topic_cols,
       index=df_metadata_dedup.index
   )

   print(doc_mat.shape)
   print(df_metadata_dedup.shape)
   print(df_doc_topic.shape)
```

```
(364, 5000)
(364, 2)
(364, 8)
```

```
]: df_doc_topic.head()
   df_doc_topic.columns
```

```
]: Index(['topic_0', 'topic_1', 'topic_2', 'topic_3', 'topic_4', 'topic_5',
         'topic_6', 'topic_7'],
       dtype='object')
```

```
]: df_metadata_dedup.head()
   df_metadata_dedup.columns
```

```
df_metadata_dedup["date"] = pd.to_datetime(df_metadata_dedup["date"])
df_metadata_dedup["date"].head()
```

```
3308228678   2026-03-01
3289985230   2026-01-04
3276785339   2025-12-01
3177718363   2025-03-17
3240212498   2025-08-17
Name: date, dtype: datetime64[ns]
```

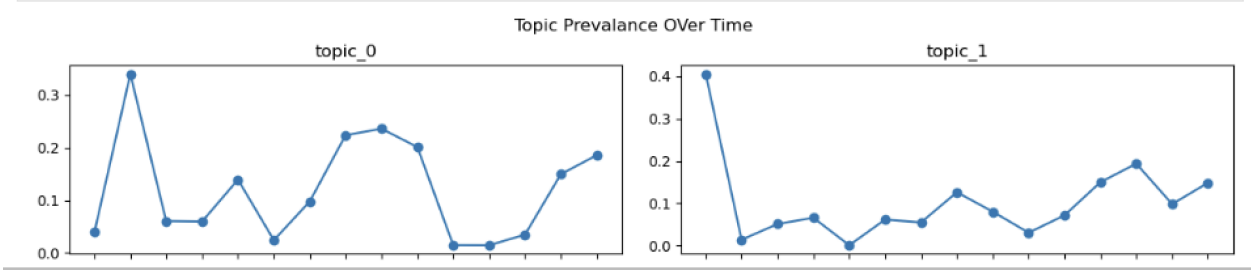
```
df_doc_topic["date"] = df_metadata_dedup["date"]
df_doc_topic["month"] = df_doc_topic["date"].dt.to_period("M")
df_doc_topic[["date", "month"]].head()
```

	date	month
3308228678	2026-03-01	2026-03
3289985230	2026-01-04	2026-01
3276785339	2025-12-01	2025-12
3177718363	2025-03-17	2025-03
3240212498	2025-08-17	2025-08

```
topic_trends = (
    df_doc_topic
    .groupby("month")[topic_cols]
    .mean()
)
topic_trends.head()
```

month	topic_0	topic_1	topic_2	topic_3	topic_4	topic_5	topic_6	topic_7
2025-01	0.039586	0.405722	0.010858	0.341398	0.146593	0.040704	0.002821	0.012319
2025-02	0.339513	0.013745	0.059356	0.400645	0.112727	0.026214	0.000367	0.047432
2025-03	0.060635	0.050862	0.005156	0.161374	0.200802	0.111141	0.000568	0.409462
2025-04	0.059720	0.065746	0.025930	0.417075	0.195844	0.233605	0.001664	0.000416
2025-05	0.139216	0.000649	0.000262	0.499413	0.227347	0.132587	0.000262	0.000262

```
# see topic prevalence over time

fig, axes = plt.subplots(4, 2, figsize=(12, 10), sharex = True)
axes = axes.flatten()

for ax, topic in zip(axes, topic_cols):
    ax.plot(topic_trends.index.astype(str), topic_trends[topic], marker="o")
    ax.set_title(topic)

plt.suptitle("Topic Prevalance Over Time")
plt.xticks(rotation=45)
plt.tight_layout()
fig.savefig(
    "topic_prevalance.png",
    dpi=300,
    bbox_inches="tight"
)

plt.show()
```



```python
topic = "topic_6"
topic_trends[topic].idxmax()
peak_month = topic_trends[topic].idxmax()
peak_month
```

```
Period('2025-10', 'M')
```

```python
df_peak = df_doc_topic[df_doc_topic["month"] == peak_month]
df_peak.shape
```

```
(15, 10)
```

```python
top_topic6_docs = (
    df_peak
    .sort_values(topic, ascending=False)
    .head(10)
)
```

```python
subset = df_metadata_dedup.loc[
    top_topic6_docs.index,
    ["date", "text"]
]
subset["preview"] = subset["text"].str[:100]
subset[["date","preview"]]
```

	date	preview
3151315396	2025-01-04	Gov. Gretchen Whitmer on Monday signed into la...
3150652785	2025-01-02	Gov. Gretchen Whitmer on Monday signed into la...

```python
idx = top_topic6_docs.index[1]
print(df_metadata_dedup.loc[idx, "date"])
print()
print(df_metadata_dedup.loc[idx, "text"][:5500])
```

```
2025-01-02 00:00:00

Gov. Gretchen Whitmer on Monday signed into law bills that touch on a range of policy areas from housing to human trafficking. Hou
sing discrimination A trio of bills signed by Whitmer will soon make it illegal for a landlord in Michigan with five or more renta
l units to discriminate against current or prospective tenants based on their source of income. Senate Bill 205 prohibits landlord
s from refusing to lease to someone based on how they earn the money they use to pay their rent. Senate Bill 206 spells out that t
hat those receiving housing assistance, veterans' benefits, Social Security and other kinds of benefits or subsidies would be cove
red by the new anti-discrimination provision. And finally, Senate Bill 207 updates the state's civil rights law to reflect the cha
nges to the state's rental regulations. All three bills passed on party-line votes with every Democratic lawmaker supporting them
against GOP opposition. Whitmer said the new laws — set to take effect in 2025 — will reduce barriers to housing. "I am committed
to growing access to affordable housing so every Michigander can raise their family in a vibrant community," Whitmer said in a sta
tement Monday. Human trafficking Three other bills — Senate Bills 515, 516 and 517 — signed by Whitmer amend the state's criminal
procedures to expand the testimony and evidence that can be used to prosecute human trafficking and prostitution. For instance, th
e legislation would allow evidence of prior acts of domestic violence, sexual assault, commercial sexual activity, human trafficki
ng or prostitution to be used in prosecutions, according to a bill summary. The bills passed on a party-line vote in the Michigan
House of Representatives with Democrats supporting the legislation, but they received wide bipartisan support in the Michigan Sena
te. Michigan was among the states with the highest number of human trafficking cases in the U.S. in 2023, according to the Nationa
l Human Trafficking Hotline. Courtroom support dogs Whitmer also signed Senate Bill 248, which raises the age for child witnesses
who can have a support dog in the courtroom from under 16 to under 18 years old. Courtroom support dogs are used to calm witnesse
s. The bill would also expand courtroom support dogs to include trained therapy dogs. The bill passed with bipartisan support in t
he Michigan House and Senate. Data center tax break Whitmer signed the first of two bills to provide tax breaks for data centers t
```

```python
topic = "topic_5"
top_n = 30

peak_months = (
    topic_trends[topic]
    .sort_values(ascending=False)
    .head(2)
    .index
)

for peak_month in peak_months:
    print("=" *60)
    print(f"TOPIC: {topic} | PEAK MONTH: {peak_month}")
    print("=" *60)

    df_peak = df_doc_topic[df_doc_topic["month"] == peak_month]

    top_docs = (
        df_peak
        .sort_values(topic, ascending=False)
        .head(top_n)
    )

    preview_df = df_metadata_dedup.loc[
        top_docs.index,
        ["date", "text"]
    ].copy()

    preview_df["preview"] = preview_df["text"].str[:300]

    display(preview_df[["date", "preview"]])
```

```
    preview_df["preview"] = preview_df["text"].str[:300]

    display(preview_df[["date", "preview"]])
```

```
============================================================
TOPIC: topic_5 | PEAK MONTH: 2025-04
============================================================
```

	date	preview
3190963319	2025-04-17	"Why are we having to go to green energy? And ...
3184869045	2025-04-02	The Big Tech boom is driving advancement throu...
3189301644	2025-04-13	Economists say the U.S. manufacturing decline ...
3189181227	2025-04-12	FILE — Shipping containers in Yangshan Port in...
3186220282	2025-04-04	1 Death toll continues to rise after Myanmar q...
3189381503	2025-04-13	Elk Grove mayor touts data centers as cash cow...

```
============================================================
TOPIC: topic_5 | PEAK MONTH: 2026-01
============================================================
```

	date	preview
3294885798	2026-01-20	LANSING — Officials from Deep Green, the Unite...

THIS ONE HAS A TYPO!!!!! Line 4 ```print ("DATE: df_metadata_dedup.loc[idx, "date"])``` I HAD AN EXTRA "s" at the end! Please don't include it!!!!

```
idx = top_docs.index[20]

full_chars = None

print("\n--- FULL ARTICLE ---")
print ("DATE:", df_metadata_dedup.loc[idx, "date"])s
print()

if full_chars is None:
    print(df_metadata_dedup.loc[idx, "text"])
else:
    print(df_metadata_dedup.loc[idx, "text"][:full_chars])
```

```
--- FULL ARTICLE ---
DATE: 2026-01-01 00:00:00

History will look back on 2025 as an important hinge year in the ongoing story of Greater Toledo. Will our population in northwest
Ohio grow and prosperity multiply? Or will we stay on a demographic doom loop gathering momentum forcing leadership into managemen
t of decline? The regional economy will benefit from the addition of a pick-up truck line at the Stellantis Jeep plant, set to add
900 jobs to Toledo's Jeep heritage, seemingly a tangible outcome of President Trump's tariff policy aimed at pressuring manufactur
ers to make their products in the United States. But it was also a year of fiscal clouds gathering. Cutbacks in state funding for
public education and competition from charter schools and state tuition vouchers to private schools forced public schools into pai
nful cuts or to seek tax increases. Toledo Public Schools ended the year planning to close six elementary schools and one high sch
ool to plug a $68 million hole. And the city of Toledo starts the year expecting to continue transferring cash from the capital im
provements fund and dipping into the rainy day fund to meet payroll. Toledo's namesake university selected a new president in 2025
```